Density Coupling

The University of Sydney

THE UNIVERSITY OF SYDNEY

Background

Coupling between Eulerian LES scheme and Lagrangian MC particle scheme is achieved by a conditional form of the equivalent enthalpy method.

$$\frac{\partial \bar{\rho} \widetilde{h_s^E}}{\partial t} + \frac{\partial \bar{\rho} \widetilde{u}_i \widetilde{h_s^E}}{\partial x_i} = -\frac{\partial}{\partial x_i}\left(\widetilde{J}_{h_s,i} + J^{\mathrm{sf}}_{h_s,i}\right) + \widetilde{W}_{h_s}$$

With the filtered source term modelled as a relaxation term towards a conditional enthalpy.

$$\widetilde{W}_{h_s} = \bar{\rho}\,\frac{\hat{h}_s - \widetilde{h_s^E}}{\tau_{\mathrm{rel}}}\,.$$

To compute the equivalent enthalpy a set equivalent species should be computed in OF. The transport equations for the set of equivalent species are analogous to the one of enthalpy and target values are computed for each of them.

Motivation

The computation of target values in the source terms can be achieved in different ways. Additionally, operations on the particle side are restricted to the computation of such target values. Hence, it makes sense to move all variables and functions related to density coupling to a sub-model.

New directories

1) Creation of new directory:

MmcFoam-4.x/src/lagrangian/mmc/submodels/Thermodynamic/ThermoPhysicalCouplingModel

2) Subdirectories including density coupling base class and submodels to compute target values.

- FlameletCurves
- KernelEstimation
- NoCoupling
- ThermoPhysicalCouplingModel

ThermoPhysicalCouplingModel.H

```
//- Conditioning Variable in density coupling used to evaluate
        const mmcVarSet& XiC_;

        //- Coupling Names
        wordList XiCNames_;

        //- Conditioning variable
        word cVarName_;

        //- Indicator is zero or one to ensure source term is zero
        volScalarField Indicator_;

        //- Thermophysical coupling (this was inside the thermoCloud)
        boolList solveEqvSpecie_;
        word primarySpecies_;
        Switch independentYEqvCurve_;
        List<label> TEqvTargetCurve_;
        List<label> YEqvTargetCurve_;
        Switch tauRelaxBlending_;
        dimensionedScalar tauRelaxStart_;
        dimensionedScalar tauRelaxTarget_;
        scalar tauRelaxDelta_;
        word tauUnits_;
```

New variables for generalised mmcFoam code. Multiple coupling (state) variables.

Members moved to density coupling sub-model.

ThermoPhysicalCouplingModel.H

```
    // Enabled
        virtual inline bool enabled() const;
```

Used to avoid computing the target enthalpy if sub-model is “none”

```
//- Find equivalent enthalpy and species curves
void findEquivalentTempSpeciesCurves();
```

This function is kept in the base class because equivalent curves are used in multiphase.

```
//- Sources

    //- Equivalent temperature and species target values
    virtual void EqvETargetValues
    (
        const mmcVarSet& Xi,
        PtrList<volScalarField>& YEqvETarget,
        volScalarField& TEqvETarget
    );
```

EqvETargetValues called in solver before evolution of Yeqv and hEqv. Defined as virtual and sub-models provide computation methods.

Kernel Estimation

A new method to compute target values introduced and named Kernel Estimation.

The traditional technique of computing target values from a precomputed set of flamelet curves is a simple an efficient method. However, this method introduces some difficulties in the simulations. The most important of them is that the target value can jump from one curve to another with quite different values causing instability in the solver.

One of the drivers for this problem is related to the use of super-cells to find the equivalent curve. For instance, LES cells that are very close yet in a different super-cell get a different target value. Also if the super-cell cover a large area and there is strong gradients of mixture fraction, the set of particles used might not be correct to select the target value (not close in composition).

Kernel Estimation

The new method uses the concept of an integral interpolant as used in smoothed particle hydrodynamics.

SPH interpolation of a quantity A, which is a function of the spatial coordinates, is based on the integral interpolant:

$$A_I(r) = \int A(r)\, W(r - r', h)\, dr$$

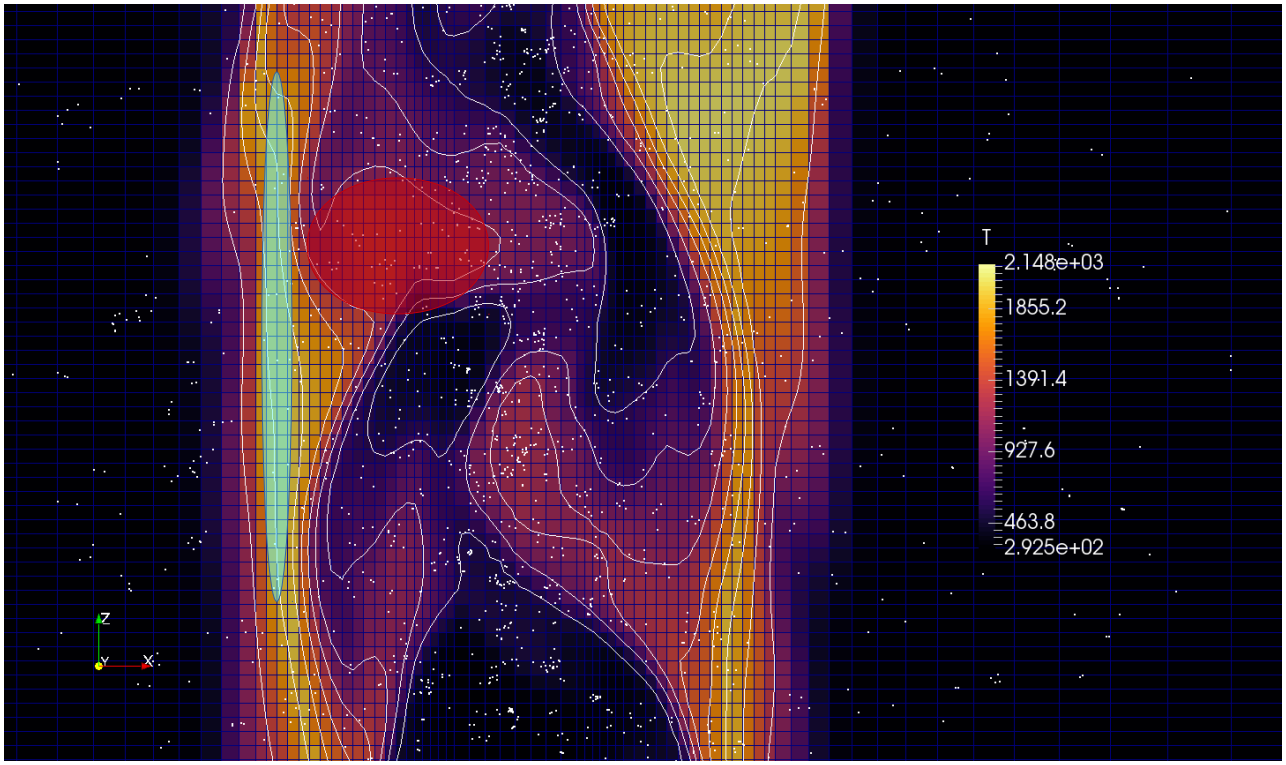
where the function W is the kernel and dr is a differential volume element . The interpolant reproduces A exactly if the kernel is a delta function. In practice,the kernels are functions which tend to the delta function as the length scale h tends to zero. They are normalized to 1 so that the constants are interpolated exactly.

The integral can then be approximated by a summation over the mass elements.

$$A_s(r) = \sum_b m_b \frac{A_b}{\rho_b} W(r - r_b, h)\, dr$$

Kernel Estimation

In SPH as in TPDF simulations the number of particles is high and neighbouring particles are very close in physical space turning them in natural neighbours. In sparse methods this is not the case and conditioning is used as in MMC mixing to pick “similar” particles. By doing this we can go much further in physical space to compute target values. In some sense this is similar to the ensembles used in conditional source estimation to solve the inverse problem.



Kernel Estimation

Kernel Estimation Interpretation

This conditioned summation integration in this method could be interpreted in different ways:

It could be seen as explicit computation of the conditional filtered average

$$\overline{f \mid \zeta}(\mathbf{x},t) = \frac{\int_D \rho(\mathbf{x}',t)\, f(\mathbf{x}',t)\; \delta[\zeta - Z(\mathbf{x}',t)]\, G(\mathbf{x}-\mathbf{x}')\, d\mathbf{x}'}{\bar{\rho}(\mathbf{x},t)\, \widetilde{P}(\zeta;\mathbf{x},t)},$$

with a Kernel in conditioning space different from a delta function.

Or as an integral interpolation in conditioning space with a kernel in physical space to reduce bias.

Either way the computation follows the same approach and once selected a suitable expression for the kernel functions, the problem is reduced to k-NN search for a set of particles to be used to compute the LES cell target value.

Kernel Estimation

Kernel Estimation computations are expensive. First, they require the computation of distances to find the k-NN particles and to be used in the kernel functions in the integral interpolation. Moreover, in the flamelet curve method the expensive operation of selection of the eqv curve is done only for supercells while the kernel operation is used in every LES cell.

To improve computational cost of the kernel estimation density coupling, the derivative approximation of SPH is used to do linear extrapolation when the next cell to be computed is close in (ζ, **x**).

$$A_s(\xi + d\xi) = A_s(\xi) + \frac{dA_s}{d\xi} \cdot \Delta\xi$$

The allowed distance in conditioning space is computed from the maximum normalised derivative out of the Eqv species and temperature and degree of refinement desired in the steep gradients. The degree of refinement is set by **C2** value and specified as a fraction or percentage of the normalised maximum derivative. In regions with small derivative, however, this choice lead to a distance in conditioning space towards infinity and a maximum value needs to be specified by **dfMax**.

Finally the distance allowed in physical space is computed from the gradient of the conditioning variable and $\Delta\xi$.

$$\Delta\vec{x} = max\left(\frac{\Delta\xi}{\nabla\xi}\right)$$

KernelEstimation.H

```
// Private data

    const fvMesh& mesh_;

    labelList Yindexes_;
    label numYEqv_;
    label numXiC_;

    const label nI_;
    const label nx_;
    const label ny_;
    const label nz_;
    const label nwt_;
    const label nT_;
    const label nDist_;
    const label nYs_;

    label nYe_;
    label nXiCs_;
    label nXiCe_;

    label numP_;

    const scalar fm_;
```

Indexes to creation a densityParticle, similar of to mixParticle in mixing models

```
const scalar fm_;
```

Kernel size in conditioning space

```
DynamicList<densParticle>
particleList_;
```

Particles in domain

```
DynamicList<densParticle>
LESList_;
```

Cells in domain

KernelEstimation.H

```
void buildParticleList();

void buildLESParticleList();


void computeTargets
(
    const mmcVarSet& XiC,
    PtrList<volScalarField>& YEqvETarget,
    volScalarField& TEqvETarget
);
```

Member functions to create list of particles and cells and function with kernel computations.

```
 //- Equivalent temperature and species target values
 virtual void EqvETargetValues
 (
     const mmcVarSet& Xi,
     PtrList<volScalarField>& YEqvETarget,
     volScalarField& TEqvETarget
 );
```

Function modified in each sub-model

KernelEstimation.H

```
const scalar fLow_;
const scalar fHigh_;
```

Kernel is biased in the boundaries of conditioning space. Since reaction negligible outside of this values and no target to compute. Also helps with significant reduction of computational time.

If conditioning space expands from 0 to 1
fLow : must be larger than fm/2
fHigh: must be smaller than 1 – fm/2

```
const word inertSpecie_;
```

to ensure mass fraction add up to 1

```
const scalar dfMax_;
const scalar C2_;
```

Used in linear extrapolation

dfMax defines maximum distance, in conditioning space, allowed to do linear extrapolation.

C2: is the fraction of the maximum gradient in conditioning space to be resolved. This controls the degree of refinement in regions with steep gradients

CloudProperties

```
//- ThermophysicalModel coupling
thermophysicalCoupling
{
    //- List of species for which equivalent
    //  mass fraction solutions are obtained
    //  on the FV grid
    CH4;
    O2;
    CO2;
    H2O;
    H2;
    condVariable        z;
    primarySpecies      CO2;

    tauRelaxBlending    false;
    tauRelaxStart       1000;
    tauRelaxDelta       2;
    tauRelax            10;
    tauUnits            timestep;
}
```

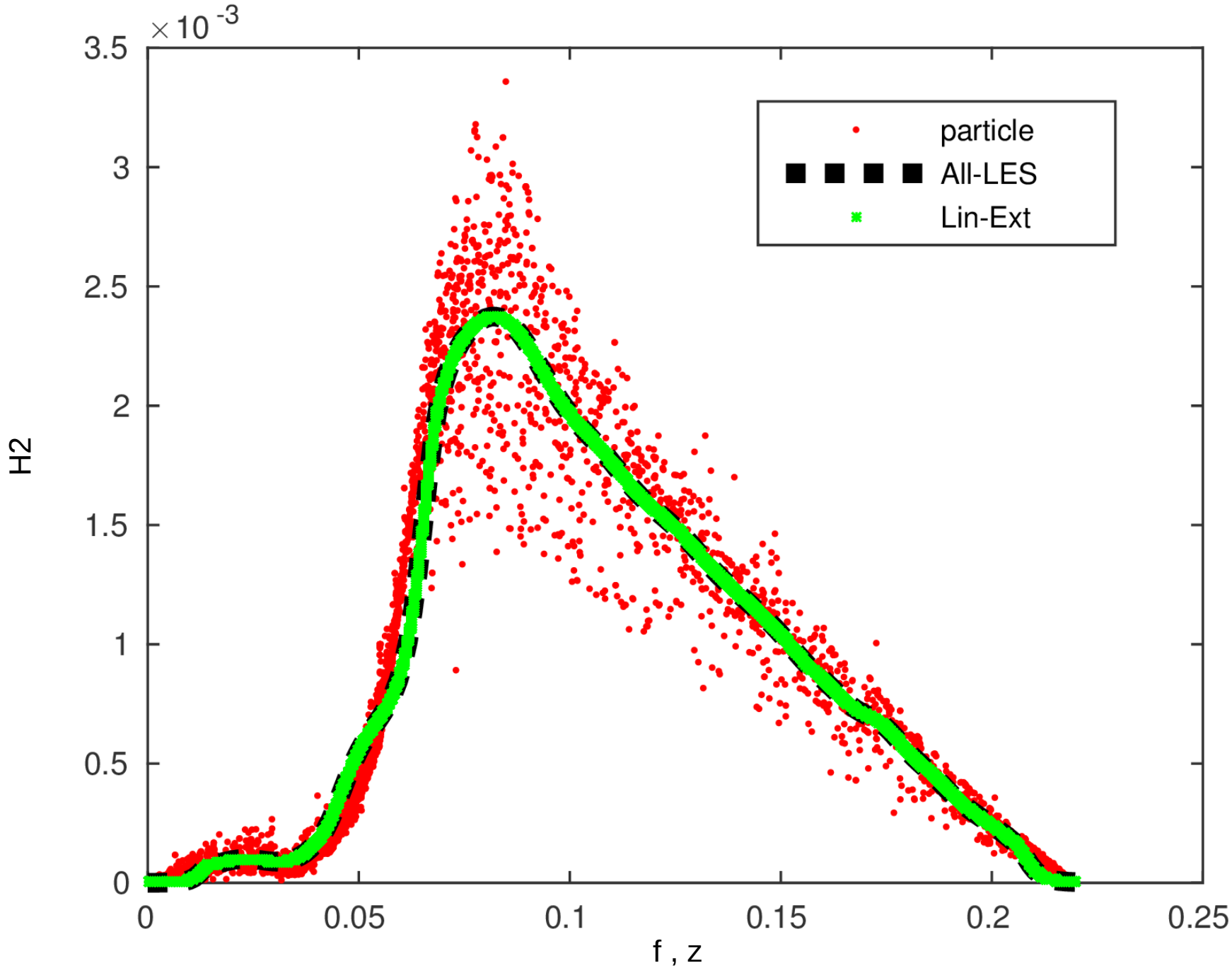
General components to set up in density coupling model.

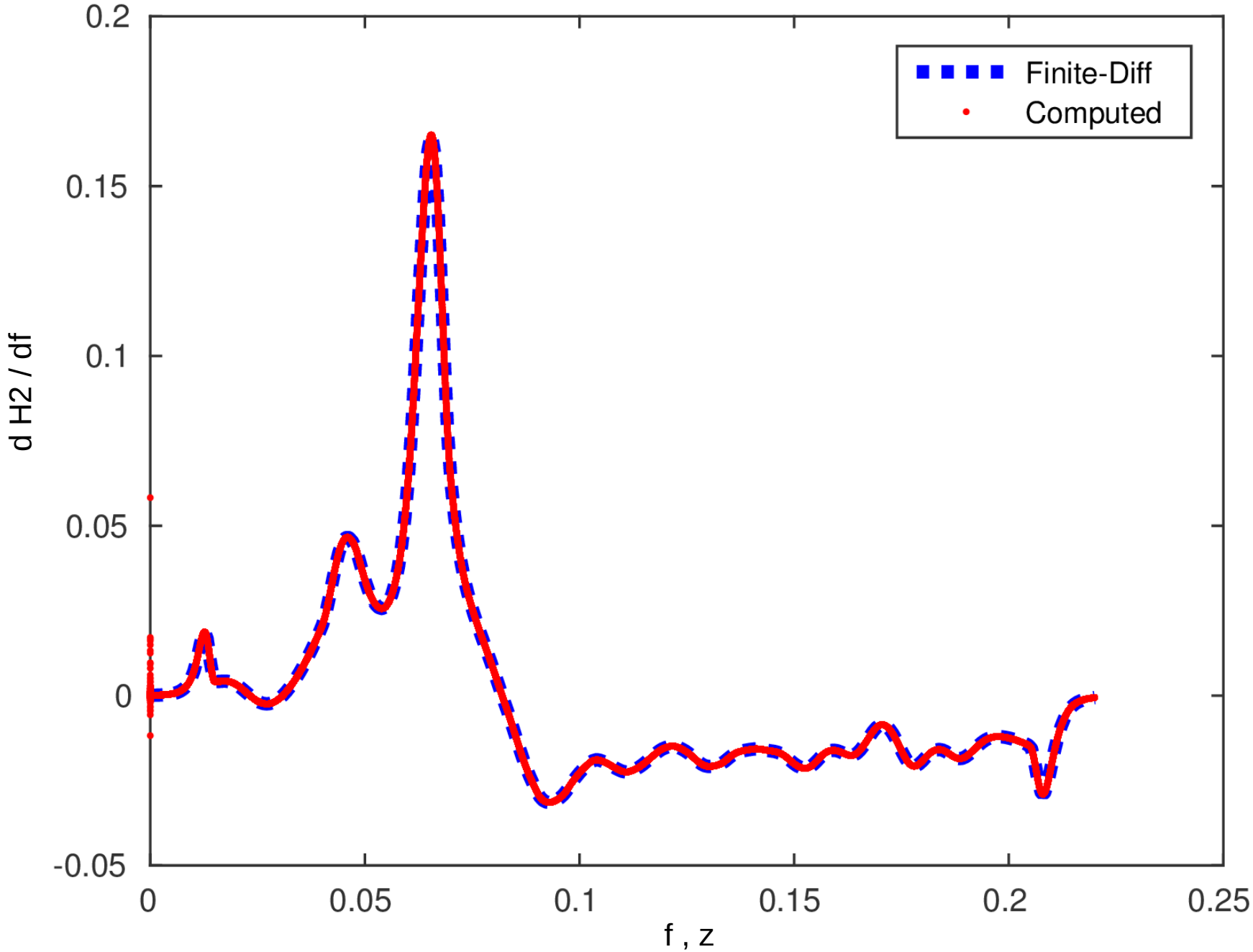
Selection of conditioning variable possible but multiple conditioning not implemented. Possible conditioning variables depend on the solver used.

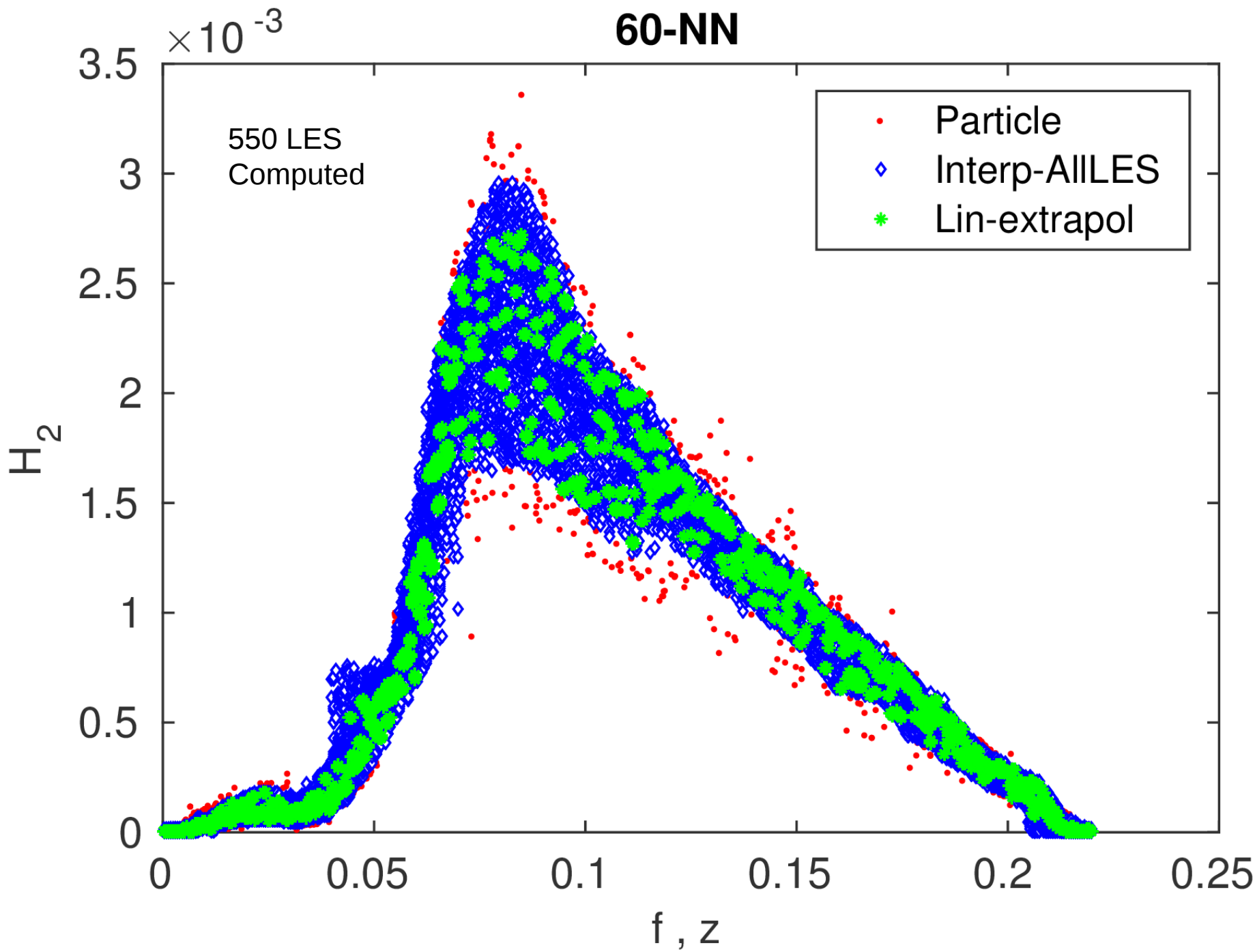
CloudProperties

```
KernelEstimationCoeffs
{
    fLow                0.03;   //lower flamability limit
    fHigh               0.19;   //upper flamability limit
    fm                  0.03;   //same as in mixing model

    dfMax               0.0075; //Maximum distance in z space allowed to do
extrapolation
    C2                  0.1;    //used to control resolution
}
```


×10 -3
3.5
3
2.5
2
1.5
1
0.5
0
H2
0
0.05
0.1
0.15
0.2
0.25
f , z
particle
All-LES
Lin-Ext


Finite-Diff
Computed
d H2 / df
f , z
0.2
0.15
0.1
0.05
0
-0.05
0
0.05
0.1
0.15
0.2
0.25


60-NN
×10 -3
3.5
3
2.5
2
1.5
1
0.5
0
H2
550 LES
Computed
Particle
Interp-AllLES
Lin-extrapol
0
0.05
0.1
0.15
0.2
0.25
f , z